Narrative Driven QA with GenAI: Enhancing Reviews, Documentation, and Epic Traceability

Shubhan Singh
STME, NMIMS Kharghar
Navi Mumbai, India
shubhan.singh312@nmims.in

Jagdish Shukla
STME, NMIMS Kharghar
Navi Mumbai, India
jagdish.shukla316@nmims.in

Ms. Tejaswini J. Chavan
STME, NMIMS Kharghar
Navi Mumbai, India
tejaswini.chavan@nmims.edu

***Abstract*—Agile QA depends on a chain of artifacts. Epics, user stories, acceptance criteria, tests, reviews, documentation: each one is supposed to carry consistent intent from product vision to delivery. In practice, LLM research has handled each link in that chain separately, treating test generation, code review, and bug reporting as independent problems. Whether LLMs can maintain semantic coherence *across* the chain—what we call cross-artifact narrative coherence—has received almost no attention. We reviewed 12 studies from 2022–2025 and classified them along four dimensions: artifact scope, narrative direction, human-LLM interaction model, and agile integration depth. The pattern is consistent. Research concentrates on single-direction, tool-level, generative work. Feedback loops are missing. Nobody has proposed metrics for cross-artifact coherence. One empirical result stands out: upstream narrative source quality measurably affects downstream artifact quality. That finding, if it generalises, has compounding implications for how teams invest in story refinement. We close with six research questions grounded in specific gaps the taxonomy exposes.**

***Index Terms*—Large Language Models, Software Quality Assurance, Agile Development, Requirements Traceability, Test Case Generation, Code Review, Narrative Coherence**

I. Introduction

Every agile project tells a story. It begins with an epic, breaks into user stories, attaches acceptance criteria, and then, ideally, those narratives shape everything that follows: the tests that verify the code, the reviews that catch its flaws, the bug reports that document its failures, and the documentation that explains its behavior to the next developer who touches it. This chain of connected artifacts is the backbone of quality assurance in agile development.

In practice, the chain breaks. Tests get written without consulting the originating story. Code reviews happen without reference to acceptance criteria. Bug reports are filed with no link to the epic they violate. The artifacts exist, but the narrative thread binding them frays under the pressure of sprint deadlines, team silos, and manual handoffs. What was supposed to be a coherent pipeline from story to delivery becomes a set of loosely related documents maintained by different people at different times. The cost is not abstract; it surfaces as escaped defects, misaligned features, rework cycles, and the slow erosion of confidence that a team is actually building what was asked for.

LLMs arrived here fast. In three years, researchers applied them to test generation [1], code review [2], [3], bug reports [4], documentation [5], and user story creation [6]. Two surveys mapped the spread [7], [8]. The field moved.

What stayed still was the question of coherence. Each QA activity has been studied as its own problem. Can the LLM generate a good test? Can it produce a useful review? Rarely has anyone asked whether these outputs hold together as a connected set, where the quality of an upstream artifact shapes what comes out downstream. The question we are interested in is not just whether LLMs produce individual QA artifacts well, but whether they can keep semantic coherence intact across the full chain: epic to story to test to review to delivery.

We reviewed 12 recent studies to examine exactly this. Rather than sort papers by QA activity (which would just reproduce the same fragmentation), we built a four-dimensional taxonomy designed to surface the connections the literature misses. Specifically, we contribute:

1) A four-dimensional taxonomy classifying LLM-assisted QA research along artifact scope, narrative direction, human-LLM interaction model, and agile integration depth.
2) A gap analysis showing that cross-artifact narrative coherence is consistently overlooked, even where the literature's own evidence suggests it matters.
3) A structured research agenda identifying six open questions for the community, grounded in specific taxonomy gaps.

The rest of the paper is laid out as follows. Section II covers background on agile QA artifacts and LLM capabilities. Section III describes our review methodology. Section IV presents the taxonomy and analysis. Section V interprets the findings. Section VI proposes the research agenda, and Section VII concludes.

II. Background

A. Narrative Artifacts in Agile QA

Agile methodologies structure development around narrative artifacts. A user story captures a unit of functionality from the user's perspective, typically following the "As a *role*, I want *goal*, so that *benefit*" template. Epics group related stories into broader product themes and serve as the high-level traceability anchor connecting individual sprints to strategic goals. Acceptance criteria define the conditions under which a

story can be marked as done. Stack them (epic, story, criteria) and you get a narrative hierarchy that is supposed to carry intent from product vision down to individual tasks.

The Quality User Story (QUS) framework by Lucassen et al. [9] formalizes what makes a story good: it should be atomic, minimal, well-formed, and include clear acceptance criteria. Stories that meet these standards constrain and inform the tests that verify them, the review criteria applied to the implementing code, and the documentation that describes the resulting behavior. Stories that fall short push ambiguity downstream. The problem scales badly. As sprints progress, stories evolve, criteria shift, and the connections between upstream narratives and downstream artifacts erode. Maintaining these links by hand is expensive and error-prone [10]. This is the gap that LLMs might address: not by generating better artifacts one at a time, but by holding the semantic thread between them.

B. LLMs in Software Engineering

That thread needs a technology that can work across artifact types. Large Language Models (transformer-based, trained on massive corpora of both text and code) are a natural candidate because they represent natural language and source code in the same learned space. For a broad treatment of LLMs in software engineering, we point the reader to Hou et al. [7] (395 studies reviewed) and Zhang et al. [8] (947 studies across 112 SE tasks). For work on LLMs and software documentation specifically, Nikoo et al. [11] provide a focused review from top SE venues.

Three capabilities matter for our purposes. First, LLMs can *generate* software artifacts from natural language input: test cases from descriptions [12], documentation from diagrams [5], user stories from requirements [6]. Second, they can *analyze* existing artifacts for quality issues, flagging code smells, bugs, or ambiguities [13]. Third, and this is the one we care about most, they can *reason across* artifact types. An LLM that reads a user story and then reviews the implementing code is doing cross-artifact reasoning in a single semantic space. Traditional static analysis tools, which operate on code structure alone, cannot do this.

C. Traceability vs. Narrative Coherence

Traceability is fundamentally a structural question: *is this artifact linked to that one?* A matrix answers it. Requirements map to tests, tests to code, code to docs; the links are binary, and recovering them is well-studied [14], [15]. But the matrix tells you nothing about whether the link is meaningful.

Narrative coherence asks the harder question: *does this artifact faithfully reflect that one?* A test case may sit in a traceability matrix alongside its user story, but if the test checks behavior the story never specified, the link is hollow. Take a concrete example. A story reads: "As a customer, I want to filter search results by price range so I can find affordable options." A traceable but incoherent test might verify that the filter UI renders correctly without ever checking that the filtered results actually respect the specified price bounds. The link exists; the semantic alignment does not.

LLMs make this distinction practical. When an LLM generates a test from a user story, it is not just creating a traceability link. It is propagating meaning from one artifact to another. When that propagation works, downstream artifacts are shaped by their source. When it fails (through hallucination, context loss, or upstream ambiguity), the downstream artifact may compile and pass, yet verify something the story never asked for.

We use the term *narrative-driven QA* throughout this paper as an analytical lens: an approach where the quality and structure of upstream narrative artifacts, from epics down through stories and acceptance criteria, actively shape the generation, evaluation, and coherence of downstream QA artifacts. We are not proposing a system here. We are looking at what exists through a different lens.

III. Review Methodology

We conducted a structured narrative review, prioritizing depth of analysis over breadth of coverage. The approach balances rigor with the realities of a focused conference paper.

A. Search Strategy

We searched six databases: IEEE Xplore, ACM Digital Library, Scopus, Google Scholar, arXiv, and Semantic Scholar. Searches ran between January and March 2026 using four thematic query blocks combined with AND operators:

- **LLM/GenAI**: "large language model" OR "generative AI" OR "GPT" OR "LLM"
- **QA/Testing**: "test case generation" OR "code review" OR "quality assurance" OR "software testing"
- **Requirements**: "user story" OR "requirements engineering" OR "acceptance criteria"
- **Traceability**: "traceability" OR "artifact consistency" OR "software documentation"

We supplemented keyword searches with forward and backward citation tracking on the most relevant initial results, using Semantic Scholar's citation graph and Connected Papers for visual cluster identification.

B. Selection Criteria

Inclusion: (1) Published or credibly pre-printed between 2022 and 2025. (2) Uses or evaluates LLMs for at least one QA-relevant activity. (3) Provides empirical evaluation or structured analysis. (4) English language.

Exclusion: (1) Tool demonstrations without evaluation. (2) Non-transformer AI approaches. (3) Papers on code generation without a QA dimension. (4) Editorials and position papers without technical contribution.

One exception to the date filter was made for Lucassen et al. [9], a foundational paper on user story quality that multiple studies in our corpus reference.

C. Corpus Composition

The initial search and screening produced roughly 90 candidates across the six databases. After full-text review and application of inclusion/exclusion criteria, we kept 12

core papers for deep taxonomic analysis and 8 supporting references for background and context, totaling 20 papers. We chose the 12 core papers to cover the spread of QA activities (requirements, testing, code review, documentation, traceability, bug reporting) while favoring studies that cross artifact boundaries or offer empirical evidence on narrative coherence.

D. Classification Procedure

Each core paper was mapped to four taxonomy dimensions described in Section IV. We did not fix these dimensions before reading. They emerged through iterative analysis and were refined as patterns became apparent. Each classification was verified against the paper's stated contributions, methodology, and evaluation scope.

IV. Taxonomy and Analysis

We now present the taxonomy, apply it to all 12 core studies, and examine what the resulting classification reveals.

A. Taxonomy Dimensions

Rather than sorting papers by QA activity, the approach taken by existing surveys [7], [8], we classify studies along four dimensions chosen to expose the *connections* (or absence of connections) between QA activities. These dimensions emerged through iterative reading and were refined as patterns became clearer.

Dimension 1: Artifact Scope. How many distinct artifact types does the study involve? We distinguish three levels: *Single* (one artifact type), *Multi* (two artifact types), and *Pipeline* (three or more in a connected chain). QA in Agile is inherently a multi-artifact problem. An epic spawns stories; stories spawn tests; tests expose bugs. Yet much of the LLM literature treats each artifact in isolation.

Dimension 2: Narrative Direction. Which way does information flow? *Forward* propagation moves from upstream artifacts (epics, stories, requirements) toward downstream ones (tests, reviews, documentation). *Backward* tracing moves in reverse, from bugs or code-level findings back toward their originating requirements. *Bidirectional* approaches support both. A healthy QA process needs flow in both directions; a bug found during testing should travel back to inform the story it came from, not simply get logged and forgotten.

Dimension 3: Human-LLM Interaction Model. What role does the LLM play? We identify four patterns: *Generator* (produces new artifacts), *Refiner* (improves existing human-written artifacts), *Linker* (establishes or recovers connections between artifacts), and *Validator* (checks artifacts for correctness or consistency). These roles carry different trust implications; generating an artifact from scratch demands more verification than refining one a human already wrote.

Dimension 4: Agile Integration Depth. How deeply does the LLM integrate with the workflow? *Tool-level* solutions plug in without changing existing processes. *Process-level* solutions alter how QA activities are sequenced, assigned, or connected. *Culture-level* solutions change team communication patterns and collaboration dynamics.

B. Classification of Studies

Table I maps each of the 12 core studies to all four dimensions.

Eight of twelve. That is not a marginal majority: it means the exceptions (C2, C7, C11) are actually the outliers in artifact scope, not the norm. The field has begun crossing artifact boundaries. But only C10 reaches pipeline scope, and C10 is a survey, not a primary investigation. No empirical study examines coherence across three or more connected artifact types.

C. Dimension-Level Analysis

1) D1: Artifact Scope: Three studies work within a single artifact type. C2 refines user stories without examining whether the improvement propagates to downstream artifacts. C7 restructures bug reports without tracing them to the stories they came from. C11 surveys QA techniques without analyzing how they connect across artifact boundaries. Useful work on its own terms, but it cannot tell us about the relationships between QA artifacts.

Among the multi-artifact studies, three deserve attention. Rahman et al. (C1) generate user stories *with* test case specifications from requirements documents, explicitly crossing the requirements-to-testing boundary [6]. Alagarsamy et al. (C3) go one step further in a specific respect: they generate test cases directly from natural language method descriptions rather than from source code [12]. That distinction matters more than it might seem. It shows that tests *can* originate from narrative input, which is a precondition for any narrative-driven QA approach. Ebert et al. (C5) build a retrieval-augmented generation pipeline drawing on code diffs, source files, and requirement tickets to produce context-aware code reviews [3]. In C5, the requirement ticket is not decoration. It actively shapes the review output. This is narrative-driven QA happening in practice, even if the authors never call it that.

C10, the lone pipeline-level entry, reviews multi-agent systems like ChatDev and AgileCoder that assign LLM agents to sequential SDLC phases [18]. These systems technically span the full pipeline, but their evaluations focus on agent collaboration mechanics, not on whether artifacts at each stage stay semantically consistent with each other. The pipeline exists. The coherence question goes unasked.

The documentation-focused studies round out the picture. Casillo et al. (C9) generate natural language use case descriptions from UML diagrams [5], flipping the typical direction: structured design artifacts produce narrative documentation rather than the reverse. Karanikolas et al. (C12) evaluate GPT-4o, Claude 3.5 Sonnet, and o3-mini for linking user documentation and API documentation to source code [20]. Together with C8 (documentation as test generation source), these studies hint at documentation playing a bigger role in the QA chain than it normally gets credit for. We come back to this in Section V.

2) D2: Narrative Direction: Five studies propagate information forward, from upstream artifacts toward downstream outputs (C1, C3, C8, C10, C12). Four work backward, from

TABLE I
TAXONOMY CLASSIFICATION OF 12 CORE STUDIES ON LLM-ASSISTED QA

ID	Study	QA Domain	D1: Scope	D2: Direction	D3: LLM Role	D4: Depth
C1	Rahman et al. [6]	Req. + Testing	Multi	Forward	Generator	Tool
C2	Zhang et al. [16]	Requirements	Single	—	Refiner	Process
C3	Alagarsamy et al. [12]	Testing	Multi	Forward	Generator	Tool
C4	Rasheed et al. [2]	Code Review	Multi	Backward	Validator+Gen.	Tool
C5	Ebert et al. [3]	Code Review	Multi	Backward	Validator	Process
C6	Ali et al. [15]	Traceability	Multi	Bidirectional	Linker	Tool
C7	Khan et al. [4]	Bug Reporting	Single	Backward	Refiner	Tool
C8	Morimoto et al. [17]	Docs + Testing	Multi	Forward	Generator	Tool
C9	Casillo et al. [5]	Documentation	Multi	Backward	Generator	Tool
C10	Rasheed, Waseem et al. [18]	Full SDLC (survey)	Pipeline	Forward	Generator	Process
C11	QA Standards Rev. [19]	QA Standards	Single	—	Mixed	Tool
C12	Karanikolas et al. [20]	Traceability	Multi	Forward	Linker	Tool

code or bugs toward originating requirements (C4, C5, C7, C9). Only C6 supports bidirectional flow, using RAG with graph indexing to link requirements and code in both directions [15].

The backward studies are individually solid but collectively narrow. Rasheed et al. (C4) deploy four specialized agents (code review, bug report, code smell, code optimization), each analyzing code and producing a different kind of feedback [2]. The bug report agent traces from code to documented issues, but the trace stops there; it does not connect the bug to the user story or acceptance criteria that the bug violates. Casillo et al. (C9) generate documentation from UML diagrams, tracing backward from design to narrative [5], but again, only one hop. No backward study chains through more than two artifact types.

The imbalance is not just a coverage gap. It has consequences. Forward propagation asks: *given this story, what should the test look like?* Backward tracing asks: *given this bug, which story did it come from?* Both are needed for a functioning QA feedback loop. Yet the combination (a system where downstream failures automatically inform and update upstream narratives) appears in one study, and only for requirements-to-code. The full cycle (epic $\rightarrow$ story $\rightarrow$ test $\rightarrow$ bug $\rightarrow$ story) has no bidirectional coverage whatsoever.

3) D3: Human-LLM Interaction Model: Generators dominate with five studies. Refiners account for two (C2, C7), validators for two (C4, C5, though C4 also generates), and linkers for two (C6, C12). The skew tells us something: the field is building, not checking. Researchers are heavily invested in whether LLMs can *produce* QA artifacts, but much less interested in whether those artifacts *align with* the narrative context they should derive from. A test suite generated by an LLM may achieve high code coverage and still miss the acceptance criteria in the originating user story. Nobody in our corpus directly measures this kind of cross-artifact semantic alignment.

4) D4: Agile Integration Depth: Nine of twelve studies operate at the tool level. They introduce an LLM capability that plugs into an existing workflow without altering it. Three engage with process-level change: C2 modifies how user stories are prepared before sprint planning, C5 restructures the code review workflow itself, and C10 reviews systems that reorganize development around multi-agent collaboration.

No study examines culture-level impact. How does LLM-assisted QA alter communication between product owners and testers? Does automated traceability reduce the need for cross-team synchronization meetings? These questions go unasked. For a methodology that explicitly values "individuals and interactions over processes and tools," that is a significant omission.

D. Cross-Dimensional Patterns

Table II cross-tabulates D1 and D2, showing where the literature concentrates and where it thins out.

TABLE II
STUDY DISTRIBUTION: D1 (SCOPE) $\times$ D2 (DIRECTION)

	Forward	Backward	Bidirect.	N/A
Single	—	C7	—	C2, C11
Multi	C1, C3, C8, C12	C4, C5, C9	C6	—
Pipeline	C10	—	—	—

The densest cell is Multi $\times$ Forward: four studies crossing one artifact boundary downstream. Multi $\times$ Backward holds three. But Pipeline $\times$ Backward and Pipeline $\times$ Bidirectional sit empty. No study traces backward through more than two artifact types, and bidirectional flow across a complete QA pipeline does not appear anywhere in the literature we reviewed.

One result from C8 matters more than the others. Morimoto and Haraguchi compared E2E test code generated from three different upstream sources: product documentation, requirement specifications, and user stories [17]. Tests from product documentation achieved higher compilation success and broader functional coverage than those from the other

two. The upstream source measurably shaped what came out downstream. If that finding holds across other contexts, then investing in narrative source quality (through LLM-assisted refinement, enrichment, or standardization) could compound across the entire QA chain. We cannot rule out that project-specific factors inflated this result, but the direction of the effect is clear.

A secondary pattern shows up when crossing D3 with D4. All three process-level studies (C2, C5, C10) involve LLMs in roles beyond pure generation: C2 uses a refiner, C5 a validator, C10 orchestrates generators within a structured agent workflow. The tool-level studies lean heavily on generators and linkers, with validators and refiners appearing less often. You probably cannot reshape a QA process without moving beyond the generator role. Producing artifacts is one thing; checking, connecting, and refining them within a workflow is another.

E. Summary of Findings

Six things stood out when we mapped the papers:

F1: The pipeline gap. No primary study examines semantic coherence across three or more connected QA artifact types. Multi-agent systems span the pipeline structurally but do not evaluate cross-artifact consistency. That gap is not subtle.

F2: Forward dominance. The 12 studies we reviewed strongly favor forward propagation. Backward tracing and bidirectional feedback loops are underrepresented, leaving the QA feedback cycle incomplete.

F3: Generator bias. LLMs are predominantly cast as artifact producers. The roles of linking for coherence and validating cross-artifact consistency receive far less attention. The field is building, not checking.

F4: Tool-level saturation. Most studies introduce LLM capabilities as drop-in tools without engaging with how QA processes or team dynamics might need to change.

F5: No coherence metrics. Individual studies measure task-specific quality (test coverage, review accuracy, report completeness), but nobody has proposed or applied metrics for cross-artifact narrative coherence. No metric exists. That is a hard stop for anyone wanting to study this rigorously.

F6: Source quality propagates. Empirical evidence (C8) demonstrates that the type and quality of the narrative source artifact directly affects the quality of generated downstream artifacts.

V. Discussion

A. Why the Fragmentation?

Read the six findings as a set and one conclusion is hard to avoid: real progress on individual tasks, near-zero progress on what connects them. The question is why.

Part of the reason is structural. Each subcommunity benchmarks against its own metrics. Test generation people measure test coverage. Code review people measure review accuracy. Documentation people measure against documentation corpora. Nobody is measuring whether a test actually reflects its originating user story. There is no benchmark for that. Until there is, cross-artifact work will not get rewarded.

The other part is that cross-artifact coherence is genuinely harder to measure. Test coverage is automatic. Semantic alignment between a test and the user story it came from requires either human judgment or an LLM-as-judge setup, both costly, both imperfect. Researchers go where the evaluation pipelines are clean. That is understandable. It is also why the blind spot persists.

The irony is hard to miss. Agile QA suffers from artifact disconnection in practice; LLM research reproduces that exact disconnection in its study designs.

B. The Narrative Coherence Argument

Finding F6 anchors a broader argument. Morimoto and Haraguchi showed that tests generated from richer documentation outperform those generated from sparse user stories [17]. The upstream narrative source, then, is not merely a starting point; it works as a quality multiplier. Improving it does not just yield a better story. It yields better tests, more targeted reviews, and more consistent documentation downstream. A stronger claim would need a larger evidence base. We are not making it. But the direction is clear.

Zhang et al. [16] offer a complementary piece: their multi-agent LLM system measurably improved story clarity and specificity. Combining the two findings suggests a pipeline worth studying. First, refine the narrative source (as in C2), then propagate it forward to generate downstream QA artifacts (as in C1, C3, C8). If the Morimoto finding holds, quality improvement at the source would cascade.

What nobody has provided yet is the backward path. When a bug surfaces, does the information travel back to update the user story? When a code review exposes a requirements gap, do the acceptance criteria get refined? The systems in our corpus treat each QA activity as a separate LLM invocation. An integrated approach, where narrative artifacts function as living documents that both inform downstream activities and absorb feedback from them, would close the loop. The empty cells in Table II mark exactly where this closure needs to happen.

There is a risk to be direct about. Khan et al. (C7) found that LLMs generating structured bug reports from unstructured input are prone to hallucination, fabricating fields like "expected behavior" with plausible but factually wrong content [4]. If narrative-driven QA depends on LLMs propagating meaning across artifact types, hallucination is not just an annoyance. It is a coherence failure that could corrupt the downstream chain. Any future system built on this idea would need human-in-the-loop validation at critical propagation points. We see no way around that requirement.

C. Documentation as a Narrative Bridge

This surprised us: documentation keeps showing up. Three studies (C8, C9, C12) all involve documentation as either an input or output in cross-artifact workflows, and collectively they suggest that documentation may serve as a more effective narrative anchor than user stories alone. Morimoto (C8) showed that product documentation produces better tests than

user stories do [17]. Casillo (C9) demonstrated that LLMs can generate readable documentation from design artifacts [5]. Karanikolas (C12) showed that LLMs can trace from documentation to code with reasonable accuracy [20].

Read the three together and they sketch a loop: documentation can be generated from design artifacts (C9), used as input to generate tests (C8), and traced back to code (C12). No single study connects all three steps, but the pieces exist. Documentation, often dismissed as a maintenance burden in agile teams, may actually be the artifact type best suited to anchor a narrative-driven QA pipeline. It carries richer context than a user story and more structured intent than raw code comments.

We are not proposing a specific system architecture. The evidence is too thin. But the *shape* of the gap, consistent across all four taxonomy dimensions, points toward a class of solutions where narrative artifacts function as semantic anchors: persistent, LLM-maintained representations that propagate context forward and absorb feedback backward. Whether such systems actually work is untested.

D. Practical Implications

For teams already adopting LLM-based QA tools, three things from this review seem worth acting on. First, narrative source quality deserves deliberate investment. A vague user story will produce vague LLM-generated tests regardless of how sophisticated the generation model is. Story refinement, whether performed by humans or by LLM agents like ALAS [16], is not overhead; it is upstream QA. Second, teams evaluating LLM QA tools should look past single-tool performance. A test generator that achieves high coverage while ignoring the acceptance criteria in the originating story introduces a quiet form of quality debt: tests that pass without actually validating what was specified. Third, documentation should not be an afterthought. Our analysis suggests that well-maintained product documentation may work as a richer input for LLM-based QA than user stories alone, and teams investing in keeping documentation current may see returns in test quality and traceability.

E. Limitations

To be precise about what we can and cannot claim: our corpus of 12 core papers is focused, not exhaustive. A broader systematic review with formal inter-rater reliability and a larger corpus would produce stronger results. The four taxonomy dimensions represent one way to organize this space; alternatives centered on model architecture, prompt strategy, or domain specificity could show different patterns. Classification along these dimensions involved judgment calls, particularly for D2 (narrative direction) and D3 (LLM role), where some papers straddle categories. We defaulted to the primary contribution described in each paper, but acknowledge that others might classify differently. The field is moving fast, and work published after our March 2026 search cutoff may change things. The taxonomy itself has not been externally validated; it is an analytical tool we derived from reading the literature, not a framework that has been tested independently.

VI. Research Agenda

Each gap we found maps directly onto something someone could actually study. We propose six questions, each tied to a specific finding from Section IV-E.

RQ1 (from F1): *Can LLMs maintain semantic coherence when generating QA artifacts across three or more artifact types in a connected pipeline?* Multi-agent systems already span the pipeline structurally. Whether the artifacts they produce at each stage remain semantically aligned, and how to measure that alignment, is unknown. A controlled experiment comparing independently generated artifacts against pipeline-generated ones, evaluated by human judges for cross-artifact consistency, would be a natural starting point.

RQ2 (from F2): *How should backward narrative tracing be designed, from bug reports through code to originating user stories and epics, using LLMs?* Forward propagation has received attention. The reverse path, tracing a downstream failure to its upstream narrative root, is largely uncharted and would complete the QA feedback loop.

RQ3 (from F3): *Can LLM-based coherence validators (systems that check whether generated tests semantically align with their originating stories) reduce defect escape rates?* We know LLMs can produce artifacts. Whether they can reliably verify cross-artifact consistency is an open question with direct practical consequences.

RQ4 (from F4): *How does LLM-assisted QA change agile team communication patterns and process dynamics?* Tool-level integration dominates what we reviewed. The process-level and culture-level implications of embedding LLMs into QA workflows have not been studied. For instance, if an LLM automatically traces a bug report to its originating story, does the tester still need to attend the sprint retrospective to explain the defect? Understanding these second-order effects matters before organizations adopt LLM-based QA at scale.

RQ5 (from F5): *What metrics can evaluate cross-artifact narrative coherence beyond binary traceability link recovery?* Right now, metrics measure individual artifact quality or link presence. Metrics for semantic alignment across artifact types, measuring *degree* of coherence rather than mere *existence* of a link, do not yet exist. Embedding-based similarity between story-test pairs, acceptance criteria coverage ratios, and LLM-as-judge consistency scores are all plausible candidates, but nobody has formally proposed or validated any of them.

RQ6 (from F6): *What is the optimal narrative source type for LLM-based downstream QA artifact generation?* Morimoto's finding that product documentation outperforms user stories and requirement specs as a test generation source raises a practical question nobody has studied systematically: which narrative artifact, at which level of detail and formality, produces the best downstream QA outcomes?

None of these questions are speculative. Each grows out of a specific, documented gap and can be pursued with existing

LLM technology and standard software engineering research methods.

VII. Conclusion

Twelve studies in, the field knows how to generate individual QA artifacts reasonably well. Whether those artifacts say the same thing as each other (whether the test reflects the story, whether the review connects to the acceptance criteria) is a question that has barely been asked. Our four-dimensional taxonomy makes the shape of that gap visible: research clusters around single-direction, tool-level generation, and leaves cross-artifact coherence, backward tracing, and process-level integration largely untouched. One empirical finding already in the literature suggests the gap matters: upstream narrative quality measurably affects what comes out downstream. The six research questions we propose are specific enough to pursue, and the gap is real enough to matter. Somebody needs to study whether the artifacts in an LLM-assisted QA pipeline actually hold together. So far, nobody has.

References

[1] J. Wang *et al.*, "Software testing with large language models: Survey, landscape, and vision," GitHub Repository: LLM-Testing/LLM4SoftwareTesting, 2024.

[2] Z. Rasheed, M. A. Sami, M. Waseem, K.-K. Kemell, X. Wang, A. Nguyen, K. Systä, and P. Abrahamsson, "AI-powered code review with LLMs: Early results," *arXiv preprint arXiv:2404.18496*, 2024.

[3] T. Ebert *et al.*, "Rethinking code review workflows with LLM assistance: An empirical study," *arXiv preprint arXiv:2505.16339*, 2025.

[4] J. Y. Khan *et al.*, "Can we enhance bug report quality using LLMs?: An empirical study of LLM-based bug report generation," in *Proc. 29th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. ACM, 2025.

[5] F. Casillo *et al.*, "Automating software documentation: Employing LLMs for precise use case description," in *Procedia Computer Science*. Elsevier, 2024.

[6] M. T. Rahman, Y. Zhu, L. Maha, C. Roy, B. Roy, and K. Schneider, "Automated user story generation with test case specification using large language model," in *Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2024, pp. 791–801, also available as arXiv:2404.01558.

[7] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[8] Q. Zhang *et al.*, "A survey on large language models for software engineering," *arXiv preprint arXiv:2312.15223*, 2024.

[9] G. Lucassen, F. Dalpiaz, J. M. E. M. van der Werf, and S. Brinkkemper, "Improving agile requirements: The quality user story framework and tool," *Requirements Engineering*, vol. 21, pp. 383–403, 2016.

[10] T. Spijkman *et al.*, "LLM-assisted requirements engineering in agile MDD," in *Proc. IEEE International Requirements Engineering Conference (RE)*. IEEE, 2025.

[11] M. S. Nikoo, S. Kochanthara, O. Babur, and M. van den Brand, "Large language models in software documentation and modeling: A literature review and findings," *arXiv preprint arXiv:2602.04938*, 2025.

[12] S. Alagarsamy, C. Tantithamthavorn, C. Arora, and A. Aleti, "Enhancing large language models for text-to-testcase generation," *Journal of Systems and Software*, 2024, also available as arXiv:2402.11910.

[13] Anonymous, "Using LLMs to enhance code quality: A systematic literature review," *Information and Software Technology*, 2025.

[14] A. D. Rodriguez, K. R. Dearstyne, and J. Cleland-Huang, "Prompts matter: Insights and strategies for prompt engineering in automated software traceability," in *Proc. IEEE International Requirements Engineering Conference Workshops (REW)*. IEEE, 2023, pp. 455–464.

[15] S. J. Ali, V. Naganathan, and D. Bork, "Establishing traceability between natural language requirements and software artifacts by combining RAG and LLMs," in *Conceptual Modeling (ER 2024)*, ser. Lecture Notes in Computer Science, vol. 15238. Springer, 2024, pp. 295–314.

[16] Z. Zhang, M. Rayhan, T. Herda, M. Goisauf, and P. Abrahamsson, "LLM-based agents for automating the enhancement of user story quality: An early report," in *Agile Processes in Software Engineering and Extreme Programming (XP 2024)*, ser. Lecture Notes in Business Information Processing, vol. 512. Springer, 2024, pp. 117–133.

[17] T. Morimoto and H. Haraguchi, "A study on the improvement of code generation quality using large language models leveraging product documentation," *arXiv preprint arXiv:2503.17837*, 2025.

[18] Z. Rasheed, M. Waseem, M. Saari, R. Rabiser, K. Systä, and P. Abrahamsson, "LLM-based multi-agent systems for software engineering: Literature review, vision and the road ahead," *arXiv preprint arXiv:2404.04834*, 2024, v4, December 2024.

[19] Anonymous, "Advancing software quality: A standards-focused review of LLM-based assurance techniques," *arXiv preprint arXiv:2505.13766*, 2025.

[20] A. Karanikolas *et al.*, "Evaluating the use of LLMs for documentation to code traceability," *arXiv preprint arXiv:2506.16440*, 2025.